

claude-windows / 662b86ab-d2e3-4c02-8a0d-ec7a9071aba8

Metadata

```
- Source: `claude-windows`  
- Kind: `claude`  
- Source file: `C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-badminton-highlight-indexer\662b86ab-d2e3-4c02-8a0d-ec7a9071aba8\subagents\workflows\wf_4849400f-9d3\agent-a049836349b8144c8.jsonl`  
- SHA-256: `5fbb4077bf313cb9fc4116416385c823310009a9d538576d11df3becd84743e9`  
- Source modified: `2026-06-21T09:27:22+00:00`  
- Imported at: `2026-07-05T16:48:27+00:00`  
- project: `wf_4849400f-9d3`  
- session_id: `662b86ab-d2e3-4c02-8a0d-ec7a9071aba8`
```

Transcript

1. user (2026-06-21T09:24:52.598Z)

You are mapping OWNER-GATED decisions for the badminton-highlight-indexer ENGINE serving track (project doc docs/COMPUTE_DECOUPLED_SERVING/README.md). Context: M0-M4 are SHIPPED (the config/input/output/device SEAM layer: deployment.profile+presets, input_backend, output base, DeviceContext). NOW the team is on M5+. The owner wants to BATCH-DECIDE every gate so the maximum independent code path is unblocked. The discipline: ONE isolated PR per milestone off origin/master, default-OFF + golden/cross-OS-gated for any core-caller change, mocked tests for anything not runnable locally (no GPU/cloud here). KEY DISTINCTION you must draw for every gate: separate (a) CODE that CAN be written + merged NOW behind default-OFF with mocked/faked tests (no owner needed) from (b) the precise OWNER PREREQUISITE (real GPU run, GCP spend, counsel/consent review, real prices, OAuth creds, a design pick). For each gate return the structured fields. Ground every claim in real file:line or doc anchors ? read the code, do not guess. Be exhaustive within your area but do not stray outside it.

YOUR AREA: M11 data-flywheel harness + the D10/D11 consent/quality gates ? what plumbing is Claude-doable vs consent-gated.

READ: docs/COMPUTE_DECOUPLED_SERVING/README.md §4.6 (the flywheel design) + D10/D11/D12, the data_pipeline/ directory (docs/data_pipeline/ and any backend code), backend/eval/experiment.py (compare / dual-eval-set), and backend/utils/workspace.py (the per-video workspace helper, default-OFF).

Determine: which of capture?reliably-store?shadow-eval?goldenize can be built NOW as default-OFF plumbing (e.g. a capture/store step into workspaces/<video_id>/, a shadow-eval harness reusing experiment.compare, a goldenize promotion stub) with faked inputs, and which strictly requires the M9 consent gate + adults-only attestation + counsel ToS (owner). Be precise about the seam between codeable plumbing and the consent decision. Report M11-plumbing and M11-consent as separate gates.

2. assistant (2026-06-21T09:24:55.094Z)

I'll start by pulling the repo and reading the assigned files to ground my analysis.

3. user (2026-06-21T09:24:58.360Z)

Your configuration specifies to merge with the ref 'refs/heads/feat/serving-m4-device-context' from the remote, but no such ref was fetched.

---BRANCH---

feat/serving-m4-device-context

4. user (2026-06-21T09:24:59.077Z)

```
README.md  
TESTING_STRATEGY.md  
architecture.svg  
runlogs  
---DATA_PIPELINE_DOCS---
```

```
GOLDEN_DATA_SHARING.md
GOLDEN_SET_IMPLEMENTATION.md
GOLDEN_SET_PHASE1_VIDEOS.md
GOLDEN_VIDEOS.md
README.md
VIDEO_RECORDING_GUIDELINES.md
---DATA_PIPELINE_CODE---
ls: cannot access 'backend/data_pipeline/': No such file or directory
./docs/data_pipeline
```

5. assistant (2026-06-21T09:25:04.243Z)

The pull failed (branch tracking ref mismatch) ? I'll proceed read-only on local state and flag it. Let me fetch origin/master for accurate base comparison and read the key files.

6. user (2026-06-21T09:25:06.945Z)

```
---LOG vs origin/master---
44c9f01 docs(serving): mark $7 tracker M4 ? + changelog (PR #283)
6932d4a feat(serving): M4 ? DeviceContext + native WASB CPU-fallback (device="auto", cuda stays strict)
```

7. user (2026-06-21T09:25:06.966Z)

```
1      """Per-video workspace path helper ? one folder per video, local?cloud identical.
2
3      See ``docs/PER_VIDEO_WORKSPACE.md``. The goal: every per-video artifact lives under
4      ONE folder keyed by the MD5 ``video_id``
5      (``backend/utils/hashing.py::compute_video_id``):
6
7          <root>/[<prefix>]/<video_id>/{source,detector,outputs,reels,logs,tmp}/...
8
9      The ``root`` is a **storage URI** (``local`` path / ``gcs://`` / ``s3://`` /
10     ``gdrive://``), so a
11     LOCAL deployment and a CLOUD (bucket) deployment share the *same relative layout* ? only
12     the root
13     differs. This module is **pure** (string/URI joins, no I/O) so it is trivially testable
14     and works
15     for every backend; the storage layer (``backend/storage/``) does the actual read/write.
16
17     Two roots by design (D3): a **durable** workspace root (may be ``gcs://``) for artifacts
18     worth
19     keeping, and a **fast local compute-scratch** mirror (always local) for frame caches /
20     extracted
21     chunks that must never be written to a cloud root. ``tmp/`` is the local-only bucket.
22
23     This is Phase 0: the helper + config fields only ? NOT yet wired into write sites (that
24     converges
25     in P2?P4, flag-gated by ``storage.single_folder_per_video``). No behaviour changes on
26     import.
27
28     """
29     from __future__ import annotations
30
31     import os
32     from dataclasses import dataclass
33
34     # Per-video subfolders. ``tmp`` is LOCAL-ONLY (frame PNGs, handoff/yolo/gemini chunks,
35     compile
36     # clips) ? huge + ephemeral, never pushed to a cloud workspace; cache-hygiene drops it.
37     SUBDIRS = ("source", "detector", "outputs", "reels", "logs", "tmp")
38     LOCAL_ONLY_SUBDIRS = ("tmp",)
39
40     def join_uri(root: str, *parts: str) -> str:
```

```

32         """Join path components onto a storage root with forward slashes.
33
34         Matches how the codebase already composes storage URIs (e.g.
35         ``f"{prefix}/weights/..."`` in
36         ``backend/worker``). Works for ``gcs://bucket``, ``s3://bucket``, ``local:///abs``
and bare
36         paths alike; empty/blank parts are skipped so ``join_uri(root, "", "x")`` ==
``join_uri(root,
37         "x")``. Forward slashes are valid path separators in Python on Windows too.
38         """
39         base = root.rstrip("/")
40         # Skip empty / whitespace-only parts; for real parts strip only slashes so spaces
inside a
41         # name (e.g. a mounted "My Drive" path) are preserved.
42         tail = "/" .join(p.strip("/") for p in parts if p and p.strip())
43         return f"{base}/{tail}" if tail else base
44
45
46     def resolve_workspace_root(workspace_root: str, output_root: str, output_dir: str) ->
str:
47         """Durable workspace root, by precedence: ``storage.workspace_root`` ?
48         ``storage.output_root`` ? ``output.output_dir``. The first non-empty wins (non-
breaking)."""
49         return (workspace_root or "").strip() or (output_root or "").strip() or (output_dir
or "output")
50
51
52     @dataclass(frozen=True)
53     class VideoWorkspace:
54         """The single folder for one video. Pure path math; build with :func:`for_video`.
55
56         ``base`` = ``<root>/[<prefix>]/<video_id>``. Subfolder accessors return URIs/paths
under it.
57         """
58
59         root: str
60         video_id: str
61         prefix: str = "workspaces"
62
63         @property
64         def base(self) -> str:
65             if self.prefix:
66                 return join_uri(self.root, self.prefix, self.video_id)
67             return join_uri(self.root, self.video_id)
68
69         def path(self, sub: str = "", name: str = "") -> str:
70             """A path under this video's folder: ``base[/sub][/name]``."""
71             return join_uri(self.base, sub, name)
72
73         @property
74         def source(self) -> str:
75             return self.path("source")
76
77         @property
78         def detector(self) -> str:
79             return self.path("detector")
80
81         @property
82         def outputs(self) -> str:
83             return self.path("outputs")
84
85         @property
86         def reels(self) -> str:
87             return self.path("reels")
88

```

```

89     @property
90     def logs(self) -> str:
91         return self.path("logs")
92
93     @property
94     def tmp(self) -> str:
95         return self.path("tmp")
96
97
98     def for_video(root: str, video_id: str, prefix: str = "workspaces") -> VideoWorkspace:
99         """Build the :class:`VideoWorkspace` for ``video_id`` under ``root`` (a storage
100         URI)."""
101         if not video_id:
102             raise ValueError("video_id is required to build a per-video workspace")
103         return VideoWorkspace(root=root, video_id=video_id, prefix=prefix)
104
105     def local_scratch_base(scratch_root: str, video_id: str) -> str:
106         """The fast LOCAL compute-scratch mirror for a video (always native FS, never a
107         cloud root).
108         Used for frame caches / extracted chunks (the ``tmp/`` bucket) during compute,
109         before durable
110         artifacts are pushed to the workspace. Uses ``os.path.join`` (OS-native separators)
111         since this
112         is always a local filesystem path.
113         """
114         if not video_id:
115             raise ValueError("video_id is required for a local scratch path")
116         return os.path.join(scratch_root or os.getcwd(), video_id, "tmp")

```

8. user (2026-06-21T09:25:07.413Z)

```

1     """Experiment harness ? A/B + shadow + side-by-side for rally detection.
2
3     This is the thin variant/experiment layer that `docs/EXPERIMENT_HARNESS.md`
4     identifies as the one missing piece: ~80% of the machinery (scoring, LOVO, the
5     no-regression gate, the pinnable classifier artifact, the persisted-candidate
6     substrate) already exists ? this module composes it. No new scoring math lives
7     here*; everything defers to:
8
9     - `backend.eval.metrics`      ? `score`, `match_intervals`, `interval_iou`
10    - `backend.eval.training`      ? `label_candidates` (y by the same IoU matcher)
11    - `backend.eval.classifier`    ? `LogisticRegression` (the pinnable JSON model)
12    - `backend.eval.gemini_refine` ? `merge_overlaps` (the blessed window merge)
13    - `backend.eval.regression`    ? `gt_hash` (label-set fingerprint)
14    - `backend.eval.calibration`   ? `pct_improvement`
15
16    The thesis (`EXPERIMENT_HARNESS.md` §2): separate the expensive, risky DETECTION
17    (per-frame signals ? run once on the GPU/WSL box, persisted into
18    `candidate_segments.stats`) from the cheap, swappable DECISION (given those
19    signals, where are the rallies). A `Variant` is a declarative re-scoring recipe;
20    given persisted candidate features it produces `List[Interval]` deterministically,
21    with no GPU and zero production risk. So most experiments are pure offline
22    re-scoring of stored data and never touch the served pipeline.
23
24    Three modes (`EXPERIMENT_HARNESS.md` §5):
25    - `compare()`      ? labeled A/B vs golden labels: per-video + LOVO-honest
26      aggregate deltas. Mirrors `distill_local`'s honest train-on-others/predict-
27      held-out loop, but variant-parameterised.
28    - `compare_unlabeled()` ? SxS on NEW footage with no ground truth: where do two
29      variants agree / diverge (A-only, B-only, agreed). The divergences are what a
30      human spot-checks (and what two highlight reels would show side by side).
31    - the promotion gate stays `run_regression.py` + `regression_baseline.json`

```

```

32         (exit 0/1/3); **rollback = flip the variant/config, never `git revert`.**
33
34     Pure + offline + unit-tested. The only DB-touching helper is
35     `load_video_candidates`, which reads persisted candidates via
36     `Database.query_candidate_segments`.
37     """
38     from __future__ import annotations
39
40     import glob
41     import json
42     import logging
43     import os
44     from dataclasses import dataclass, field
45     from datetime import datetime, timezone
46     from hashlib import sha1
47     from typing import Any, Callable, Dict, List, Optional, Sequence
48
49     from backend.eval.calibration import pct_improvement
50     from backend.eval.classifier import LogisticRegression
51     from backend.eval.gemini_refine import merge_overlaps
52     from backend.eval.metrics import Interval, match_intervals, score
53     from backend.eval.regression import gt_hash
54     from backend.eval.training import label_candidates
55     from backend.eval import eval_stats as _stats          # C1: CIs + paired sign test
56     from backend.eval import segmentation_metrics as _segm  # C4: F1@k + segment-count
57     ratio
58     scores
59     logger = logging.getLogger(__name__)
60
61     # Where a comparison run appends one reproducible record. Tracked location (NOT under
62     # the gitignored output/) so the leaderboard survives a clean checkout, mirroring
63     # regression.DEFAULT_BASELINE_PATH.
64     DEFAULT_LEADERBOARD_PATH = os.path.join("eval_baselines", "experiment_leaderboard.json")
65     _METRICS = ("precision", "recall", "f1", "mean_iou")
66
67
68     class ExperimentError(ValueError):
69         """A variant cannot be evaluated as configured (e.g. a learned variant asked to
70         score an unlabeled video with no pinned model, or a gate referencing an absent
71         feature)."""
72
73
74     # ----- Variant
75
76
77     @dataclass
78     class Variant:
79         """A declarative re-scoring recipe ? NOT a code branch (`EXPERIMENT_HARNESS.md` §4).
80
81         A variant turns one video's persisted candidate windows + features into rally
82         intervals. Every knob defaults to the **control** behaviour (no gate, no learned
83         filter), so a variant that merely *carries* a new, disabled cue is byte-identical
84         to control ? the core no-regression guarantee.
85
86         Fields:
87         - ``segmenter`` / ``config_overrides`` / ``cue_flags`` ? informational provenance
88           for the leaderboard and for selecting the served path (the existing
89           `segmenter_registry` + `IndexingConfig` do the actual selection in production).
90           New cues are recorded here default-OFF.
91         - ``min_crossings`` ? decision-gate **Gate A**: keep only windows whose persisted
92           ``net_crossings`` feature is >= this. 0 = off. This is the eval-only gate from
93           `rally_gate.py` expressed as a re-scoring knob (kills service-feed / held-
94 shuttle

```

```

94         windows ? see `MULTI_SIGNAL_FUSION_PLAN.md` §3.1).
95     - ``min_players`` ? decision-gate **Gate B** (the future person cue): keep windows
96       whose persisted ``players_in_court`` is >= this. 0 = off (no-op until the person
97       cue persists that feature).
98     - ``classifier_model`` ? path to a frozen `LogisticRegression` JSON. When set,
99       `compare()` scores videos with the SHIPPED model as-is (no per-fold training);
100       required for `compare_unlabeled` on a learned variant.
101     - ``feature_names`` ? the persisted features the learned filter consumes. When set
102       WITHOUT ``classifier_model``, `compare()` trains a fresh model per LOVO fold
103       (honest) ? the recipe, not a pinned artifact.
104     - ``threshold`` ? classifier probability cutoff. ``merge_gap`` ? post-filter
105       overlap-merge gap (seconds), the same `gemini_refine.merge_overlaps` default.
106     """
107
108     name: str
109     segmenter: str = "wasb_hybrid"
110     config_overrides: Dict[str, Any] = field(default_factory=dict)
111     cue_flags: Dict[str, bool] = field(default_factory=dict)
112     min_crossings: int = 0
113     min_players: int = 0
114     classifier_model: Optional[str] = None
115     feature_names: Optional[List[str]] = None
116     threshold: float = 0.5
117     merge_gap: float = 1.0
118     # C2 / threshold-in-LOVO (default OFF ? unchanged behaviour). When set, the
operating
119     # point is chosen on the TRAINING fold, never the held-out one ? fixing the
first-A/B
120     # failure where a fixed 0.5 zeroed out a small-candidate video.
121     tune_threshold: bool = False          # pick the F1-best threshold on the train fold
122     calibrate: Optional[str] = None       # None | "platt" | "isotonic" ? calibrate
proba first
123
124     def is_learned(self) -> bool:
125         """True if this variant applies a learned classifier (pinned model or
recipe)."""
126         return self.classifier_model is not None or bool(self.feature_names)
127
128     def to_dict(self) -> dict:
129         return {
130             "name": self.name,
131             "segmenter": self.segmenter,
132             "config_overrides": self.config_overrides,
133             "cue_flags": self.cue_flags,
134             "min_crossings": self.min_crossings,
135             "min_players": self.min_players,
136             "classifier_model": self.classifier_model,
137             "feature_names": self.feature_names,
138             "threshold": self.threshold,
139             "merge_gap": self.merge_gap,
140             "tune_threshold": self.tune_threshold,
141             "calibrate": self.calibrate,
142         }
143
144     @classmethod
145     def from_dict(cls, d: dict) -> "Variant":
146         known = {f for f in cls.__dataclass_fields__} # type: ignore[attr-defined]
147         return cls(**{k: v for k, v in d.items() if k in known})
148
149     def spec_hash(self) -> str:
150         """Stable 12-char hash of the recipe ? identifies the variant in the leaderboard
151         and makes a result reproducible. The pinned **control** variant always hashes
the
152         same, so a regression is always traceable to a recipe change, never to drift."""
153         blob = json.dumps(self.to_dict(), sort_keys=True, default=str)

```

```

154         return sha1(blob.encode()).hexdigest()[:12]
155
156
157 #: The pinned, frozen baseline: candidates as detected, merged, no gate, no learned
158 #: filter. Always the comparison reference; "rollback" = make this the served variant.
159 CONTROL = Variant(name="control")
160
161
162 # ----- persisted candidates
163
164
165 @dataclass
166 class VideoCandidates:
167     """One video's persisted detection, ready for offline re-scoring.
168
169     ``intervals`` are the candidate windows; ``features`` is column-major
170     (``feature_name`` -> per-candidate value``, each list aligned to ``intervals``);
171     ``gts`` are golden labels (None for new/unlabeled footage). Build one from the DB
172     with ``load_video_candidates``, or directly in tests.
173     """
174
175     name: str
176     intervals: List[Interval]
177     features: Dict[str, List[float]] = field(default_factory=dict)
178     gts: Optional[List[Interval]] = None
179
180
181 def _is_fusion_schema(candidates: Sequence[Dict[str, Any]]) -> bool:
182     """True iff every candidate row carries the per-cue feature dicts the golden-feature
loaders
183     read (``shuttle`` + ``person``; ``audio`` stays optional ? it is added later by
184     ``fusion_audio.augment`` and older JSONs predate it).
185
186     The velocity-windowing / distill corpus-growth path persists a LEANER
187     ``*_golden_features.json`` row (``start``/``end`` + velocity stats, no cue dicts)
for clips
188     that never went through fusion feature extraction. The loaders that build cue-rich
189     ``VideoCandidates`` from these files ? ``ablation.load_videos``,
``fusion_compare.load_videos``
190     and ``fusion_audio.load_videos_with_audio`` ? SKIP those rather than crash on
``c["shuttle"]``,
191     so a MIXED corpus (some fusion files, some windowing-only) loads the fusion subset
cleanly.
192     An empty candidate list is not a fusion file either.
193
194     Single source of truth for the three loaders (consolidated 2026-06-18 after PRs
#209/#210
195     each landed an independent copy)."""
196     return bool(candidates) and all("shuttle" in c and "person" in c for c in
candidates)
197
198
199 def load_golden_videos(
200     features_dir: str,
201     *,
202     shuttle_columns: Sequence[str],
203     person_columns: Sequence[str],
204     audio_columns: Sequence[str] = (),
205     audio_policy: str = "omit",
206     skip_logger: logging.Logger = logger,
207     skip_message: str,
208     no_audio_error: str = "{path} has no audio features",
209 ) -> List["VideoCandidates"]:
210     """Shared golden-feature corpus loader behind the three structurally-identical
loaders

```

```

211         (``ablation.load_videos`` / ``fusion_compare.load_videos`` /
``fusion_audio.load_videos_with_audio``).
212
213     Reads every FULL-FUSION-SCHEMA ``*_golden_features.json`` in ``features_dir`` into a
214     ``VideoCandidates`` (intervals + the caller-supplied feature columns + golden gts).
Leaner
215     velocity-windowing-only files (no ``shuttle``/``person`` cue dicts ? the schema the
216     corpus-growth path persists) are SKIPPED, not crashed on (see
``_is_fusion_schema``), so a
217     MIXED corpus loads the fusion subset cleanly.
218
219     The column lists are ALWAYS supplied by the caller ? this loader never hardcodes
columns. The
220     three callers differ ONLY in:
221
222     - ``audio_policy`` ? how the audio block is handled, checked BEFORE reading any
audio column
223     so a windowing-only file is skipped before either branch applies:
224     * ``"omit"`` ? no audio columns are loaded (``audio_columns``
ignored);
225     * ``"tolerate-missing"`` ? audio columns load, a missing block defaults them
to 0.0;
226     * ``"require"`` ? a fusion file lacking audio raises ``SystemExit``
(via
227     ``no_audio_error`` formatted with ``path``), mirroring "run ``--augment``
first".
228     - ``skip_logger`` / ``skip_message`` ? the per-caller logger + ``%s``-templated
info line
229     emitted when a windowing-only file is skipped (kept byte-identical per caller).
230     ""
231     if audio_policy not in ("omit", "tolerate-missing", "require"):
232         raise ValueError(
233             f"audio_policy must be 'omit'/'tolerate-missing'/'require', got
{audio_policy!r}")
234     videos: List[VideoCandidates] = []
235     for path in sorted(glob.glob(os.path.join(features_dir, "*_golden_features.json"))):
236         with open(path, encoding="utf-8") as f:
237             d = json.load(f)
238             cands = d["candidates"]
239             if not _is_fusion_schema(cands):
240                 skip_logger.info(skip_message, os.path.basename(path))
241                 continue
242             if audio_policy == "require" and "audio" not in cands[0]:
243                 raise SystemExit(no_audio_error.format(path=path))
244             intervals = [(float(c["start"]), float(c["end"])) for c in cands]
245             features: Dict[str, List[float]] = {}
246             for fn in shuttle_columns:
247                 features[fn] = [float(c["shuttle"][fn]) for c in cands]
248             for fn in person_columns:
249                 features[fn] = [float(c["person"][fn]) for c in cands]
250             if audio_policy == "tolerate-missing":
251                 for fn in audio_columns:
252                     features[fn] = [float((c.get("audio") or {}).get(fn, 0.0)) for c in
cands]
253             elif audio_policy == "require":
254                 for fn in audio_columns:
255                     features[fn] = [float(c["audio"][fn]) for c in cands]
256             gts = [(float(a), float(b)) for a, b in d["gts"]]
257             videos.append(VideoCandidates(name=d["name"], intervals=intervals,
features=features, gts=gts))
258
259     return videos
260
261
262 def _feature_column(vc: VideoCandidates, name: str) -> List[float]:
263     """Persisted feature column, defaulting missing/short columns to 0.0 (matches

```



```

264     `training.extract_yolo_features`, which lets a sparse/old row still vectorise)."""
265     col = vc.features.get(name)
266     n = len(vc.intervals)
267     if col is None:
268         logger.warning("variant references feature %r absent from %s ? treating as 0.0",
269                        name, vc.name)
270         return [0.0] * n
271     if len(col) != n:
272         raise ExperimentError(
273             f"feature {name!r} has {len(col)} values but {vc.name} has {n} candidates")
274     return [float(x) for x in col]
275
276
277     def _feature_matrix(vc: VideoCandidates, feature_names: Sequence[str]) ->
List[List[float]]:
278         cols = [_feature_column(vc, name) for name in feature_names]
279         return [list(row) for row in zip(*cols)] if cols else [[] for _ in vc.intervals]
280
281
282     def _gate_mask(variant: Variant, vc: VideoCandidates) -> List[bool]:
283         """Boolean keep-mask from the decision gates (Gate A net-crossings, Gate B
284         players)."""
285         n = len(vc.intervals)
286         mask = [True] * n
287         if variant.min_crossings > 0:
288             col = _feature_column(vc, "net_crossings")
289             mask = [m and (col[i] >= variant.min_crossings) for i, m in enumerate(mask)]
290         if variant.min_players > 0:
291             col = _feature_column(vc, "players_in_court")
292             mask = [m and (col[i] >= variant.min_players) for i, m in enumerate(mask)]
293         return mask
294
295     def predict(variant: Variant, vc: VideoCandidates,
296                model: Optional[LogisticRegression] = None,
297                threshold: Optional[float] = None,
298                calibrator: Optional[Callable[[float], float]] = None) -> List[Interval]:
299         """Variant prediction: gate by persisted features, optionally filter by a learned
300         model, then merge. Pure and deterministic.
301
302         ``model`` (an already-fitted `LogisticRegression`) is supplied by `compare` per LOVO
303         fold; if omitted and the variant pins ``classifier_model``, that frozen model is
304         loaded. A learned variant with neither a supplied nor a pinned model raises ? there
305         is nothing to score with. ``threshold``/``calibrator`` (both fitted on the TRAINING
306         fold) override the variant defaults when the C2 / threshold-in-LOVO path supplies
307         them.
308
309         """
310         mask = _gate_mask(variant, vc)
311         if variant.feature_names:
312             if model is None:
313                 if variant.classifier_model is None:
314                     raise ExperimentError(
315                         f"variant {variant.name!r} is learned but has no model to predict "
316                         f"with (pass a fitted model or set classifier_model)")
317                 model = LogisticRegression.load(variant.classifier_model)
318             proba = [float(p) for p in model.predict_proba(_feature_matrix(vc,
319 variant.feature_names))]
320             if calibrator is not None:
321                 proba = [calibrator(p) for p in proba]
322             thr = variant.threshold if threshold is None else threshold
323             mask = [m and (proba[i] >= thr) for i, m in enumerate(mask)]
324             kept = [iv for iv, keep in zip(vc.intervals, mask) if keep]
325             return merge_overlaps(kept, gap_tol=variant.merge_gap)

```

```

325     def _calibrate_and_tune(variant: Variant, model: LogisticRegression,
326                             train_videos: Sequence[VideoCandidates], iou: float
327                             ) -> "tuple[Optional[Callable[[float], float]],
Optional[float]]":
328         """Fit a calibrator and/or pick the operating threshold on the TRAINING fold only
329         (C2 + threshold-in-LOVO). Returns ``(calibrator, threshold)`` ? either may be None
330         (use the variant default). Honest: never looks at the held-out video.
331         """
332         calibrator: Optional[Callable[[float], float]] = None
333         if variant.calibrate:
334             raw: List[float] = []
335             y: List[int] = []
336             for vc in train_videos:
337                 raw.extend(float(p) for p in
338                             model.predict_proba(_feature_matrix(vc, variant.feature_names or
339                                                                 [])))
340                 y.extend(label_candidates(vc.intervals, vc.gts or [], iou))
341             if variant.calibrate == "platt":
342                 calibrator = fit_platt(raw, y)
343             elif variant.calibrate == "isotonic":
344                 calibrator = fit_isotonic(raw, y)
345             else:
346                 raise ExperimentError(f"unknown calibrate={variant.calibrate!r} (use
347                                     'platt'/'isotonic')")
348             threshold: Optional[float] = None
349             if variant.tune_threshold:
350                 best_t, best_f1 = variant.threshold, -1.0
351                 for k in range(1, 20):
352                     # grid 0.05..0.95 on the train
353                     t = round(0.05 * k, 2)
354                     f1s = [score(predict(variant, vc, model=model, threshold=t,
355                                         calibrator=calibrator),
356                               vc.gts or [], iou).f1 for vc in train_videos]
357                     mean_f1 = sum(f1s) / len(f1s) if f1s else 0.0
358                     if mean_f1 > best_f1:
359                         best_f1, best_t = mean_f1, t
360                     threshold = best_t
361             return calibrator, threshold
362
363         # ----- variant scoring
364
365     def _train(variant: Variant, videos: Sequence[VideoCandidates], iou: float) ->
LogisticRegression:
366         """Fit the variant's classifier on the pooled candidates of ``videos`` (labels via
367         the
368         evaluator's own IoU matcher). The honest-LOVO training step from `distill_local`.
369         """
370         X: List[List[float]] = []
371         y: List[int] = []
372         for vc in videos:
373             if vc.gts is None:
374                 raise ExperimentError(f"cannot train: {vc.name} has no golden labels")
375             X.extend(_feature_matrix(vc, variant.feature_names or []))
376             y.extend(label_candidates(vc.intervals, vc.gts, iou))
377         if not X:
378             raise ExperimentError("cannot train a learned variant on zero candidates")
379         return LogisticRegression().fit(X, y, variant.feature_names)
380
381     def evaluate_variant(videos: Sequence[VideoCandidates], variant: Variant,
382                         iou: float = 0.5, honest: bool = True) -> Dict[str, Any]:
383         """Score a variant on a labeled corpus. Returns per-video Scores + predictions + a
384         mean aggregate.

```

```

383 Prediction policy:
384 - **static variant** (no learned filter): predict each video directly ? no
training,
385 so no leakage; per-video scoring is already honest.
386 - **learned, pinned model** (``classifier_model`` set): score every video with the
387 SHIPPED model. ? optimistic if those videos were in its training set ? use this
to
388 report "how the shipped artifact does here", not for the promotion decision.
389 - **learned recipe** (``feature_names``, no pinned model) + ``honest=True``: LOVO
?
390 train on the OTHER videos, predict the held-out one. The number to trust.
391 - learned recipe + ``honest=False``: train on all, predict each (overfit; sanity
only).
392 """
393 for vc in videos:
394     if vc.gts is None:
395         raise ExperimentError(f"{vc.name} has no golden labels; use
compare_unlabeled")
396
397     per_video: List[Dict[str, Any]] = []
398     predictions: Dict[str, List[Interval]] = {}
399
400     recipe_lovo = bool(variant.feature_names) and variant.classifier_model is None
401     pinned_model = (LogisticRegression.load(variant.classifier_model)
402                     if variant.classifier_model is not None else None)
403     all_model = None
404     if recipe_lovo and not honest:
405         all_model = _train(variant, videos, iou)
406
407     for i, vc in enumerate(videos):
408         cal: Optional[Callable[[float], float]] = None
409         thr: Optional[float] = None
410         if recipe_lovo and honest:
411             others = [v for j, v in enumerate(videos) if j != i]
412             if not others:
413                 raise ExperimentError("LOVO needs >=2 videos; pass honest=False or a
pinned model")
414             model: Optional[LogisticRegression] = _train(variant, others, iou)
415             if variant.tune_threshold or variant.calibrate: # operating point from
the train fold
416                 assert model is not None # _train never returns
None
417                 cal, thr = _calibrate_and_tune(variant, model, others, iou)
418             elif recipe_lovo: # honest=False ? one model over all
videos
419                 model = all_model
420             else: # static variant or pinned model
421                 model = pinned_model
422             preds = predict(variant, vc, model=model, threshold=thr, calibrator=cal)
423             assert vc.gts is not None # guarded at function top
424             sc = score(preds, vc.gts, iou)
425             per_video.append({"video": vc.name, "num_pred": len(preds), **{m: getattr(sc, m)
for m in _METRICS}})
426             predictions[vc.name] = preds
427
428     aggregate = {m: (sum(p[m] for p in per_video) / len(per_video) if per_video else
0.0)
429                  for m in _METRICS}
430     return {
431         "variant": variant.name,
432         "spec_hash": variant.spec_hash(),
433         "is_learned": variant.is_learned(),
434         "honest_lovo": recipe_lovo and honest,
435         "per_video": per_video,
436         "aggregate": aggregate,

```

```

437         "predictions": predictions,
438     }
439
440
441     def _ci_block(eval_v: Dict[str, Any]) -> Dict[str, Any]:
442         """Per-metric mean + bootstrap CI over the per-video scores (C1 ? never a bare
443         mean)."""
444         return {m: _stats.mean_with_ci([p[m] for p in eval_v["per_video"]]) for m in
445         _METRICS}
446
447     def _segmentation_block(videos: Sequence[VideoCandidates], eval_v: Dict[str, Any]) ->
448     Dict[str, Any]:
449         """Mean F1@k + segment-count ratio across videos (C4 ? the over-segmentation tIoU
450         hides)."""
451         gts_by_name = {v.name: (v.gts or []) for v in videos}
452         overlaps = _segm.DEFAULT_OVERLAPS
453         f1k: Dict[float, List[float]] = {ov: [] for ov in overlaps}
454         ratios: List[float] = []
455         for name, preds in eval_v.get("predictions", {}).items():
456             gts = gts_by_name.get(name, [])
457             got = _segm.f1_at_overlaps(preds, gts, overlaps)
458             for ov in overlaps:
459                 f1k[ov].append(got[ov])
460             ratios.append(_segm.segment_count_ratio(preds, gts))
461
462     def _mean(xs: List[float]) -> float:
463         return sum(xs) / len(xs) if xs else 0.0
464
465     out: Dict[str, Any] = {"f1@{int(ov * 100)}": _mean(vals) for ov, vals in
466     f1k.items()}
467     out["segment_count_ratio"] = _mean(ratios)
468     return out
469
470     def honest_summary(videos: Sequence[VideoCandidates], eval_a: Dict[str, Any],
471     eval_b: Dict[str, Any]) -> Dict[str, Any]:
472         """C1 + C4 honest reporting layered over a compare() pair (additive,
473         EXPERIMENT_HARNESS $6).
474
475         ``per_video`` lists are video-aligned (``evaluate_variant`` iterates ``videos`` in
476         order),
477         so ``paired_delta_f1`` pairs B?A by video ? the promotion-grade view: a lift is real
478         when
479         the ?F1 CI clears 0 *and* the sign test agrees, not when a bare mean ticked up.
480         """
481         a_f1 = [p["f1"] for p in eval_a["per_video"]]
482         b_f1 = [p["f1"] for p in eval_b["per_video"]]
483         return {
484             "variant_a_ci": _ci_block(eval_a),
485             "variant_b_ci": _ci_block(eval_b),
486             "paired_delta_f1": _stats.paired_delta_ci(a_f1, b_f1),
487             "segmentation": {"variant_a": _segmentation_block(videos, eval_a),
488                             "variant_b": _segmentation_block(videos, eval_b)},
489         }
490
491     def compare(videos: Sequence[VideoCandidates], variant_a: Variant, variant_b: Variant,
492     iou: float = 0.5, honest: bool = True, honest_stats: bool = True) ->
493     Dict[str, Any]:
494         """Offline labeled A/B (`EXPERIMENT_HARNESS.md` $5.1). Scores both variants on the
495         same
496         golden corpus and reports the **B ? A** deltas, per video and in aggregate.
497
498         The deltas use the existing `metrics.score` (per video) +
499         `calibration.pct_improvement`;

```

```

491     learned variants are scored LOVO-honestly. The result is a self-contained record fit
for
492     `record_experiment` ? variant specs + hashes + the label-set fingerprint travel with
it.
493
494     ``honest_stats`` (default True) **adds** a ``"honest"`` block ? per-metric bootstrap
CIs, a
495     paired ?F1 CI + exact sign test (C1), and F1@k + segment-count ratio (C4) ?
*alongside* the
496     existing bare-mean fields (additive: nothing existing changes). Set False for the
legacy
497     shape. This is the measurement-honesty wiring (`ANALYZERS_AND_RECONCILERS.md`
§13/C1+C4): a
498     bare LOVO mean at n?6 is not trustworthy on its own.
499     ""
500     if len(videos) < 1:
501         raise ExperimentError("compare needs at least one labeled video")
502     eval_a = evaluate_variant(videos, variant_a, iou, honest)
503     eval_b = evaluate_variant(videos, variant_b, iou, honest)
504
505     delta = {m: eval_b["aggregate"][m] - eval_a["aggregate"][m] for m in _METRICS}
506     delta_pct = {m: pct_improvement(eval_a["aggregate"][m], eval_b["aggregate"][m]) for
m in _METRICS}
507
508     by_video_a = {p["video"]: p for p in eval_a["per_video"]}
509     per_video_delta = [
510         {"video": p["video"], **{m: p[m] - by_video_a[p["video"]][m] for m in _METRICS}}
511         for p in eval_b["per_video"]]
512     ]
513
514     # One fingerprint over the whole label set ? detects "the deltas were measured
against
515     # different labels" the same way regression.gt_hash guards the no-regression
baseline.
516     all_gts: List[Interval] = [iv for vc in videos for iv in (vc.gts or [])]
517
518     result: Dict[str, Any] = {
519         "iou": iou,
520         "label_set_hash": gt_hash(all_gts),
521         "num_videos": len(videos),
522         "variant_a": eval_a,
523         "variant_b": eval_b,
524         "delta": delta,
525         "delta_pct": delta_pct,
526         "per_video_delta": per_video_delta,
527     }
528     if honest_stats:
529         result["honest"] = honest_summary(videos, eval_a, eval_b)
530     return result
531
532
533     # ----- SxS on unlabeled video
534
535
536     def compare_unlabeled(video: VideoCandidates, variant_a: Variant, variant_b: Variant,
537         agree_iou: float = 0.5) -> Dict[str, Any]:
538         """Side-by-side on NEW footage with no ground truth (`EXPERIMENT_HARNESS.md` §5.2).
539
540         With no labels there is no lift to measure ? instead this surfaces where the two
541         variants **agree vs diverge**, which is exactly what a human reviews (and what two
542         highlight reels would show side by side):
543
544         - ``agreed`` ? windows both variants found (matched at ``agree_iou``);
545         - ``a_only`` ? A fired, B suppressed (B's added precision, if A was over-firing);
546         - ``b_only`` ? B fired, A did not (B's new detections ? real recall or new FPs:

```

```

547         the human / a later golden label adjudicates).
548
549     Both variants must be predictable without labels: a learned variant therefore needs
550     a
551     pinned ``classifier_model`` (you cannot train on an unlabeled video). Reuses
552     ``metrics.match_intervals`` ? no new matching logic.
553     """
554     preds_a = predict(variant_a, video)
555     preds_b = predict(variant_b, video)
556     mr = match_intervals(preds_a, preds_b, agree_iou)
557
558     agreed = [{"a": preds_a[m.pred_index], "b": preds_b[m.gt_index], "iou": m.iou}
559               for m in mr.matches]
560     agree_iou = [m.iou for m in mr.matches]
561     a_only = [preds_a[i] for i in mr.unmatched_pred_indices]
562     b_only = [preds_b[i] for i in mr.unmatched_gt_indices]
563
564     def _dur(ivs: List[Interval]) -> float:
565         return sum(e - s for s, e in ivs)
566
567     return {
568         "video": video.name,
569         "agree_iou": agree_iou,
570         "variant_a": variant_a.name,
571         "variant_b": variant_b.name,
572         "num_a": len(preds_a),
573         "num_b": len(preds_b),
574         "num_agreed": len(agreed),
575         "mean_agree_iou": (sum(agree_iou) / len(agree_iou)) if agree_iou else 0.0,
576         "duration_a": _dur(preds_a),
577         "duration_b": _dur(preds_b),
578         "agreed": agreed,
579         "a_only": a_only,
580         "b_only": b_only,
581     }
582
583     # ----- leaderboard / DB
584
585     def record_experiment(result: Dict[str, Any], path: str = DEFAULT_LEADERBOARD_PATH,
586                          timestamp: Optional[str] = None) -> Dict[str, Any]:
587         """Append a compact, reproducible record of a ``compare()`` result to the JSON
588         leaderboard (``EXPERIMENT_HARNESS.md`` §6: experiments are records). Generalises
589         ``regression_baseline.json``; deliberately the size of a baseline file, not a service
590         (§9). Returns the row written. ``timestamp`` is injectable for deterministic tests.
591         """
592         row: Dict[str, Any] = {
593             "timestamp": timestamp or datetime.now(timezone.utc).isoformat(),
594             "iou": result.get("iou"),
595             "label_set_hash": result.get("label_set_hash"),
596             "num_videos": result.get("num_videos"),
597             "variant_a": {"name": result["variant_a"]["variant"],
598                           "spec_hash": result["variant_a"]["spec_hash"],
599                           "aggregate": result["variant_a"]["aggregate"]},
600             "variant_b": {"name": result["variant_b"]["variant"],
601                           "spec_hash": result["variant_b"]["spec_hash"],
602                           "aggregate": result["variant_b"]["aggregate"]},
603             "delta": result.get("delta"),
604         }
605
606         # Record the honest ?F1 (CI + sign test) when present, so the leaderboard carries
607         the
608         # promotion-grade evidence, not just the bare-mean delta (additive ? C1).
609         honest = result.get("honest") or {}
610         if honest.get("paired_delta_f1"):

```

```

610         row["honest_delta_f1"] = honest["paired_delta_f1"]
611     entries: List[Dict[str, Any]] = []
612     if os.path.isfile(path):
613         with open(path, encoding="utf-8") as f:
614             entries = json.load(f)
615     entries.append(row)
616     os.makedirs(os.path.dirname(path) or ".", exist_ok=True)
617     with open(path, "w", encoding="utf-8") as f:
618         json.dump(entries, f, indent=2)
619     logger.info("recorded experiment %s vs %s (?f1=%0.3f) -> %s",
620               row["variant_a"]["name"], row["variant_b"]["name"],
621               (result.get("delta") or {}).get("f1", 0.0), path)
622     return row
623
624
625     def load_video_candidates(db: Any, video_id: str, *, detector: Optional[str] = None,
626                             gts: Optional[List[Interval]] = None,
627                             feature_names: Sequence[str] = ()) -> VideoCandidates:
628         """Bridge persisted detection ? `VideoCandidates` via
629         `Database.query_candidate_segments`.
630
631         ``feature_names`` selects which persisted features to lift out of each row's
632         ``stats``
633         JSON (``"duration"`` is derived from the window bounds). This is the only DB-
634         touching
635         function; the comparison itself is pure and offline. Exercised owner-side on a real
636         DB,
637         but works against any `Database` (including an in-memory one in tests).
638         """
639         rows = db.query_candidate_segments(video_id, detector=detector)
640         intervals: List[Interval] = [(float(r["start_time"]), float(r["end_time"])) for r in
641         rows]
642         features: Dict[str, List[float]] = {}
643         for name in feature_names:
644             if name == "duration":
645                 features[name] = [float(r["end_time"]) - float(r["start_time"]) for r in
646                 rows]
647             else:
648                 features[name] = [float((r.get("stats") or {}).get(name, 0.0)) for r in
649                 rows]
650         return VideoCandidates(name=video_id, intervals=intervals, features=features,
651         gts=gts)
652
653

```

9. assistant (2026-06-21T09:25:10.130Z)

Now let me read the core serving doc §4.6 (flywheel) and the D10/D11/D12 gates, plus the data_pipeline README.

10. user (2026-06-21T09:25:13.034Z)

```

161 docs/COMPUTE_DECOUPLED_SERVING/README.md
---
36:| D9 | **Edge auth = Cloudflare Access** (reuse the site's existing CF). Lock the Cloud Run
**ingress to the CF proxy** + **verify the `Cf-Access` JWT signature** (so a direct hit to the
Cloud Run URL can't spoof the identity header). | owner 2026-06-21 | M9. One identity system. |
37:| D10 | **Free-tier "data-for-reels" trade:** free reels in exchange for **training rights**
(uploaded footage + derived trajectories/labels); **paid tier = no-train** (privacy is the paid
feature). **Carve-outs:** train ONLY on attested **ADULT** footage (minor footage ? reel yes,
training pool **NO** ? DPDP/GDPR need verifiable parental consent); train on **DERIVED** data +
**auto-delete raw** ; **ToS counsel-reviewed** ; live training-on-uploads **gated to M9** (public
signup). Truly-local needs no DPA. | owner 2026-06-21 | M9 + D12; ties [[paper-coauthor-aim]] /
PERSONALIZATION flywheel. |
38:| D11 | **Quality floor: Gemini handoff ON** for cloud-serving until the owned model clears
the #172 ship-gate (e.g. ?0.70 multi-court) ? then flip via a **Launch Report** ; the owned model

```

runs in **shadow/eval** meanwhile. (Gemini = serving/inference ONLY, never a training source ? CLAUDE.md guardrail.) | owner 2026-06-21 | M7 + D12. |

39:| D12 | **Data-flywheel harness is IN SCOPE** (owner add, Q5):**capture the (consented, adult) uploaded video + the Gemini reel/rally output ? reliably store** in the per-video workspace/bucket ? **run shadow evals** (owned-vs-Gemini, dual-eval-set) ? **promote good ones to the golden TRAINING set** (human-reviewed labels) so the owned model climbs toward the D11 floor (then Gemini flips off). | owner 2026-06-21 | new **M11**; ties D10 (consent) + ``data_pipeline/`` + the eval/experiment harness. |

53:| **ADR-011** | DECOUPLED scope = M0?M2+M3.5; **deferred M3 (serving) + M4 (billing/cost-guard)**; overrode the **"rent nothing / not a service"** posture | **This project realizes M3/M4.** Carries §06's owner-gated gates: **DPA/consent** for non-owner uploads, collector ``md5`` keying, WASB-C3. |

94:### 4.6 The data-flywheel harness (D12 / M11) ``[design]``

95:Gemini-on (D11) ships good reels now; this harness is **how we earn the right to flip Gemini off**. On a **consented, adult** cloud-serving job (the D10 trade), the pipeline additionally:

96:1. **Captures** the source upload + the Gemini reel + the rally output (intervals/report) into the per-video workspace (``workspaces/<video_id>``), instead of the no-consent auto-delete path.

97:2. **Reliably stores** them durably in the bucket (the consented-retention bucket; raw video kept only for the training pool, not the no-consent path).

98:3. **Runs shadow evals** ? the owned model scores the same video alongside Gemini (``backend/eval/experiment.compare`` + the dual-eval-set, ``[[eval-dual-set-regression]]``) ? tracks the owned-vs-Gemini gap per job (the live read on "is the floor reachable yet?").

99:4. **Promotes to golden** ? captured videos that pass review become **golden TRAINING rows** (human-reviewed labels via the annotation flow ? ``data_pipeline/``), growing the corpus ? the owned model climbs to the D11 floor ? flip Gemini off via a Launch Report.

100:This is the consent?capture?eval?goldenize **flywheel** (the cloud realization of the PERSONALIZATION contribution model): every consented free reel makes the owned model better. Reuses ``data_pipeline/``, ``backend/eval/``, and the M9 consent gate; **adults-only**, raw-retention gated by the D10 consent.

106:- This does not resolve **WASB-C3** (sharing a trained model) or the **DPA/consent** gate ? both remain owner/counsel-gated before public, non-owner serving.

122:| M9 | **Edge auth** (Cf-Access extraction) + per-tenant scoping on ``user_id``; DPA/consent gate wiring | M7 | ? owner (privacy/counsel) | ? | ? |

123:| M10 | ``byo_worker`` / zero-infra-BYO ? **design-only, DEFERRED** | M8,M9 | ? explicit owner approval | ? | ? |

124:| **M11** | **Data-flywheel harness (D12):** on a consented adult job, **capture** the upload + Gemini reel/rally output ? **reliably store** under the per-video workspace ? **shadow-eval** owned-vs-Gemini (dual-eval-set / ``experiment.compare``) ? **promote** good ones to the golden TRAINING set (human-reviewed labels via the annotation flow). Closes the loop so the owned model climbs to the D11 floor. Reuses ``data_pipeline/`` + ``backend/eval/`` + the consent gate (M9). | M7,M9 | ? owner (consent/privacy) | ? | ? |

125:| **M12** | ``gdrive-api`` (OAuth) StorageBackend (D14 Northstar):**cloud reads the source from + writes the stitched reel back to the user's Google Drive next to the source** (per-user OAuth consent) ? the mobile/cloud-native Drive round-trip. Distinct from the local mount backend; slots into the ``StorageBackend`` ABC. | M7,M9 | ? owner (OAuth/consent) | ? | ? |

130:**? The 5 gating questions are LOCKED (owner, 2026-06-21) ? see §2 D7?D12:**

134:5. ? **DPA/consent ? D10** (free-tier data-for-reels trade; adults-only; derived-data; counsel-gated to M9).

135:6. ? **Quality floor ? D11** + the **data-flywheel harness D12 / M11** (Gemini-ON until the owned model clears the floor; capture?store?eval?goldenize meanwhile).

148:- [] Cost ledger (M8) makes GPU + **stitch** + egress cost visible per job; edge auth (M9) gates multi-tenant.

149:- [] **Data-flywheel harness (M11)** captures?stores?shadow-evals?goldenizes consented adult jobs; the owned model is on a measured path to the D11 floor.

11. assistant (2026-06-21T09:25:15.404Z)

Now let me read the full §4.6 region and the data_pipeline README to understand the goldenize/annotation flow, plus check how workspaces and storage are wired today.

12. user (2026-06-21T09:25:17.379Z)

85 - **Env auto-detect SUGGESTS**, the user **CONFIRMS** (D15 ? no surprise execution):**the env gives a *proposed* default** (``K_SERVICE`` ? cloud-serving; ``CI=true`` ? cpu-mock; else


```

`torch.cuda.is_available()` ? truly-local), but the user explicitly confirms or overrides it
? a GPU owner may still pick cloud, and cloud (= spend) is NEVER entered silently. The
active profile is always surfaced (logged/shown). Per-profile startup checks fail/warn fast
(cloud-serving without GCS creds/GPU ? loud error before any job).
86     - The worker (`cmd_infer`/`cmd_train`) is already profile-agnostic (reads
`config.storage`, accepts `--config`/`--set`) ? no worker rewrite.
87
88     ### 4.4 Compute placement ? and why stitch runs on Cloud Run `[design]`
89     DETECT on GPU everywhere; WINDOW CPU-pure everywhere; STITCH is real CPU work. In
cloud-serving, stitch is co-located in the Cloud Run job so `gpu_active_seconds` (detect) +
`wall_seconds` (detect+stitch) + `bytes_out` (reel egress) land on one metered invocation ?
the per-job cost ledger. Running stitch on a laptop would leave that cost
invisible/unattributed ? exactly what the owner wants to avoid. (Stitch stays synchronous in
the GPU job for the common one-reel case; a queued CPU-only stitch fallback only if compiles get
bursty ? the job table + `claim_due_job` already supports both.)
90
91     ### 4.5 Reuse map ? most of the seams already exist `[verified]`
92     Exists: the 3-stage core; `_resolve_runner` (the single compute seam);
`StubDetectorRunner` (`replay_csv` = byte-exact $0 anchor); `StorageBackend` ABC + 4 backends +
`StorageRef.parse_uri`; the worker CLI (fetch/put, `--config`/`--set`); the async job control
plane (`jobs` table, `claim_due_job` CAS); `ExecutionPolicy WAIT_AND_RESUME`/`JobWaiting`
(carries to Cloud Run dispatch verbatim); the chunked-upload router (`<2GB`); `skip_ai_handoff`
(LocalNoAI); storage fakes; golden fixtures + experiment harness. Partial: per-video
workspace helper (default-OFF); `DeviceContext` (designed); edge-auth (v4 `user_id` columns
exist). Net-new: the `deployment` section; `input_backend`; configurable output base (kill
hardcoded `output/`); Cloud Run dispatch shim + container image; cost ledger + pricebook; edge-
auth header extraction; startup env detection.
93
94     ### 4.6 The data-flywheel harness (D12 / M11) `[design]`
95     Gemini-on (D11) ships good reels now; this harness is how we earn the right to flip
Gemini off. On a consented, adult cloud-serving job (the D10 trade), the pipeline
additionally:
96     1. Captures the source upload + the Gemini reel + the rally output
(intervals/report) into the per-video workspace (`workspaces/<video_id>/`), instead of the no-
consent auto-delete path.
97     2. Reliably stores them durably in the bucket (the consented-retention bucket; raw
video kept only for the training pool, not the no-consent path).
98     3. Runs shadow evals ? the owned model scores the same video alongside Gemini
(`backend/eval/experiment.compare` + the dual-eval-set, [[eval-dual-set-regression]]) ? tracks
the owned-vs-Gemini gap per job (the live read on "is the floor reachable yet?").
99     4. Promotes to golden ? captured videos that pass review become golden TRAINING
rows (human-reviewed labels via the annotation flow ? `data_pipeline/`), growing the corpus ?
the owned model climbs to the D11 floor ? flip Gemini off via a Launch Report.
100    This is the consent?capture?eval?goldenize flywheel (the cloud realization of the
PERSONALIZATION contribution model): every consented free reel makes the owned model better.
Reuses `data_pipeline/`, `backend/eval/`, and the M9 consent gate; adults-only, raw-
retention gated by the D10 consent.
101
102    ## 6. Honest limits ? what this does NOT do / prove
103    - A green CI suite proves plumbing + determinism (window+stitch) + profile-parity
for $0 ? it does NOT prove field quality, nor that real-GPU detection is good (that's owner-
side, real-video).
104    - Cross-GPU detection is NOT byte-identical ? parity is a window-level claim, not a
CSV-byte claim.
105    - The cloud `0.1.0-l4` weights are n=2 plumbing-grade ? cloud-serving ships with
Gemini handoff ON until the owned model clears a floor (open Q).
106    - This does not resolve WASB-C3 (sharing a trained model) or the DPA/consent
gate ? both remain owner/counsel-gated before public, non-owner serving.
107
108    ## 7. Deliverables & progress tracker ? source of truth
109    Legend: ? Todo · ? In progress · ? Done · ? Gated. One small PR per row, off
`origin/master`, no stacking. Default-OFF where it touches core callers.
110
111    | ID | Deliverable | Depends | Gated? | Status | PR |
112    |----|-----|-----|-----|-----|----|

```

```

113 | M0 | This design doc + project dir + `runlogs/` + ADR-014 | ? | owner sign-off | ? |
**? APPROVED 2026-06-21** |
114 | M1 | `deployment.profile` + `compute_target` config + preset bundles +
precedence/auto-detect; unit tests (no behaviour change, profile='' default) | M0 | safe | ? |
[#279](https://github.com/Khelsutra/badminton-highlight-indexer/pull/279) |
115 | M2 | `input_backend`/`input_root`; wire main CLI/API input through
`StorageRef.parse_uri` (local=identity); storage-fake tests | M1 | safe | ? |
[#280](https://github.com/Khelsutra/badminton-highlight-indexer/pull/280) |
116 | M3 | Configurable output/scratch base ? kill hardcoded `output/`
(`trajectory_hybrid:237,313`, `yolo_hybrid:407`, `gemini`); **DEFAULT stays `output/`**; golden
+ stub-E2E byte-identical | M1 | ? default-OFF + Launch Report on flip | ? |
[#282](https://github.com/Khelsutra/badminton-highlight-indexer/pull/282) |
117 | M4 | `DeviceContext` + WASB **CPU-fallback** ; unified healthcheck/device wiring | M1 |
safe (cuda default unchanged) | ? | [#283](https://github.com/Khelsutra/badminton-highlight-
indexer/pull/283) |
118 | M5 | **truly-local profile** end-to-end (preset + startup checks); first committed
**runlog** (truly-local sample run) | M2,M3,M4 | owner real-GPU | ? | ? |
119 | M6 | Cloud Run GPU **dispatch shim** (control plane ? job via storage ? re-claim) +
**containerized worker image** (ffmpeg+CUDA+WASB+fonts); mocked-dispatch tests | M5 | ? owner
(GCP spend) | ? | ? | ? |
120 | M7 | **cloud-serving profile** e2e on L4: upload`<2GB` + gdrive/bucket ? detect+stitch
on Cloud Run ? reel via signed URL; **DEPLOY_REPORT** (posterity, sample runs + contact sheet) |
M6 | ? owner (spend) | ? | ? | ? |
121 | M8 | Per-job **cost ledger** (v_ migration: `owner_id, gpu_active_seconds,
wall_seconds, bytes_out, cost_usd, cost_breakdown_json`) + **pricebook** + aggregation +
`/api/.../cost` | M7 | ? owner (billing) | ? | ? | ? |
122 | M9 | **Edge auth** (Cf-Access extraction) + per-tenant scoping on `user_id`;
DPA/consent gate wiring | M7 | ? owner (privacy/counsel) | ? | ? | ? |
123 | M10 | `byo_worker` / zero-infra-BYO ? **design-only, DEFERRED** | M8,M9 | ? explicit
owner approval | ? | ? | ? |
124 | **M11** | **Data-flywheel harness (D12)**: **on a consented adult job, **capture** the
upload + Gemini reel/rally output ? **reliably store** under the per-video workspace ? **shadow-
eval** owned-vs-Gemini (dual-eval-set / `experiment.compare`) ? **promote** good ones to the
golden TRAINING set (human-reviewed labels via the annotation flow). Closes the loop so the
owned model climbs to the D11 floor. Reuses `data_pipeline/` + `backend/eval/` + the consent
gate (M9). | M7,M9 | ? owner (consent/privacy) | ? | ? | ? |
125 | **M12** | **`gdrive-api` (OAuth) StorageBackend (D14 Northstar)**: **cloud reads the
source from + writes the stitched reel **back to the user's Google Drive next to the source**
(per-user OAuth consent) ? the mobile/cloud-native Drive round-trip. Distinct from the local
mount backend; slots into the `StorageBackend` ABC. | M7,M9 | ? owner (OAuth/consent) | ? | ? | ? |
126
127 **Testing strategy is a first-class deliverable** ?
[ `TESTING_STRATEGY.md` ](TESTING_STRATEGY.md). **Run logs / sample runs for posterity** ?
[ `runlogs/` ](runlogs/).
128
129 ## 8. Open questions ? owner *(blocking-gates vs nice-to-knows)*

```

13. user (2026-06-21T09:25:17.540Z)

```

1 # data_pipeline/ ? data collection · annotation · golden sets · sharing (**NO ML**)
2
3 The **data layer**: everything about *getting, recording, annotating, assembling, and
sharing/ingesting
4 the rally **data** ? deliberately **isolated from the ML work**. Segmentation,
stitching, models, and
5 the model-eval framework live at `docs/` top level + `COMPUTE_DECOUPLED_SERVING/`, **not
here**.
6
7 **In scope (what belongs here)**:
8 - [ `VIDEO_RECORDING_GUIDELINES.md` ](VIDEO_RECORDING_GUIDELINES.md) ? how to capture
footage (collection).
9 - [ `GOLDEN_SET_IMPLEMENTATION.md` ](GOLDEN_SET_IMPLEMENTATION.md) ·
[ `GOLDEN_SET_PHASE1_VIDEOS.md` ](GOLDEN_SET_PHASE1_VIDEOS.md) ·
[ `GOLDEN_VIDEOS.md` ](GOLDEN_VIDEOS.md) ? golden-set creation + which videos.
10 - [ `GOLDEN_DATA_SHARING.md` ](GOLDEN_DATA_SHARING.md) ? how the data/artifacts are shared

```

```

+ ingested (collector / OneDrive flow).
11
12     **Deliberately NOT here:**
13     - **ML / model work** ? segmentation + stitching design, the rally-quality eval
framework (`../EVALUATION.md`, `../RALLY_QUALITY_RESEARCH.md`, the R-series), and the
regression-fixtures harness (`../GOLDEN_REGRESSION_FIXTURES.md`, which is ML-eval). Where a doc
here references the ML side, it is a **pointer** (`../X.md`), not ML detail.
14     - **The annotation *vendor* engagement** ? the Roora docs (`ROORA_LABELING_GUIDELINES`,
`GOLDEN_SET_VENDORING`) are archived under [ `../archives/roora-vendor/` ](../archives/roora-
vendor/).
15
16     > Boundary established 2026-06-21 (owner). The data pillar is reusable knowledge about
the *inputs*; the ML pillar is the *processing*. Keep them separate.
17

```

14. assistant (2026-06-21T09:25:20.356Z)

Now let me understand the existing golden-set / training-row persistence flow and the annotation flow, plus how the storage layer and config are structured. This grounds what plumbing already exists vs needs building.

15. user (2026-06-21T09:25:23.100Z)

```

=== golden_features writers (the goldenize sink) ===
backend/eval/ablation.py
backend/eval/experiment.py
backend/eval/fusion_audio.py
backend/eval/fusion_compare.py
backend/eval/fusion_golden.py
backend/eval/rally_seg_eval.py
backend/eval/serve_contrast.py
backend/eval/__pycache__/ablation.cpython-313.pyc
backend/eval/__pycache__/experiment.cpython-313.pyc
backend/eval/__pycache__/fusion_audio.cpython-313.pyc

=== annotation flow ===
backend/eval/ablation.py
backend/eval/ablation_pdf.py
backend/eval/annotations.py
backend/eval/audio/e1_probe.py
backend/eval/audio/__pycache__/e1_probe.cpython-313.pyc
backend/eval/calibrate_local.py
backend/eval/calibrate_wasb.py
backend/eval/calibration.py
backend/eval/distill_local.py
backend/eval/eval_partial_labels.py

=== storage backends ===
__init__.py
__pycache__
base.py
gcs.py
gdrive.py
local.py
ref.py
s3.py

=== config storage section ===
__init__.py
__pycache__
loader.py
models.py
presets.py

```

16. user (2026-06-21T09:25:27.565Z)

```
=== golden_features WRITERS (json.dump to *_golden_features) ===
backend/eval/fusion_golden.py:149:         out = os.path.join(out_dir,
f"{spec['name']}_golden_features.json")
```

17. user (2026-06-21T09:25:29.989Z)

```
=== storage models: consent / retention / capture / training fields? ===
45:     # decoded frame cache) are deleted automatically ? they are pure intermediates,
77:     # (the ~GBs/video disk hog) are deleted automatically ? pure intermediates, regenerable
317:    # --- Per-video workspace (docs/PER_VIDEO_WORKSPACE.md) ---
318:    # One folder per video keyed by the MD5 video_id:
<root>/[<workspace_prefix>]/<video_id>/...
322:    single_folder_per_video: bool = False
323:    # Durable workspace root URI. "" => precedence: output_root, then output.output_dir.
324:    workspace_root: str = ""
326:    # videos/ labels/ weights/ (e.g. gs://bucket/workspaces/<video_id>/). "" = root
directly.
327:    workspace_prefix: str = "workspaces"
--- presets ---
41:# these bundles ? ``input_backend`` (M2) and the configurable output/workspace base (M3,
```

18. assistant (2026-06-21T09:25:33.305Z)

Let me read the golden_features writer in fusion_golden.py (the goldenize sink) and the storage config section around workspace, plus check the `serve\_contrast.py` (owned-vs-Gemini contrast ? likely the shadow-eval seam).

19. user (2026-06-21T09:25:34.747Z)

```
120         "label": int(labels[i]),
121     })
122     return {
123         "name": spec["name"], "fps": fps, "frame_width": fw,
124         "candidates": candidates,
125         "gts": [[float(a), float(b)] for a, b in gts],
126     }
127
128
129     def run(manifest_path: str, out_dir: str, frame_skip: int = 3, iou: float = 0.5,
130           fresh: bool = False, smallest_first: bool = False) -> List[str]:
131         with open(manifest_path, encoding="utf-8") as f:
132             specs = json.load(f)
133         if smallest_first:
134             # Process small videos first for quick feedback (the slow full match lands
last);
135             # the LOVO result is order-independent. Proxy size by the trajectory CSV (one
row/frame).
136             specs = sorted(specs, key=lambda s: (os.path.getsize(s["trajectory"])
137                                                  if os.path.isfile(s.get("trajectory", ""))
else 0))
138             os.makedirs(out_dir, exist_ok=True)
139             # Run-telemetry black box: a long GPU run can be OS-killed with no traceback, so
keep a tiny
140             # rotating runlog of velocity + machine health for the post-mortem. Constructing it
must never
141             # break extraction, so guard the build and fall back to no telemetry on any error.
142             telem: Optional[RunTelemetry]
143             try:
144                 telem = RunTelemetry(os.path.join(out_dir, "run_telemetry.jsonl"),
label="fusion_golden")
145             except Exception: # noqa: BLE001 - telemetry wiring must never break the run it
observes
```

```

146         telem = None
147     written: List[str] = []
148     for vi, spec in enumerate(specs, 1):
149         out = os.path.join(out_dir, f"{spec['name']}_golden_features.json")
150         # Resumable: skip a video whose JSON already exists (a re-run picks up where a
prior
151         # run left off instead of re-detecting). --fresh forces a redo.
152         if not fresh and os.path.isfile(out):
153             print(f"[fusion_golden] ({vi}/{len(specs)}) {spec['name']}: already
extracted "
154                 f"-> {out} (use --fresh to redo)", flush=True)
155             written.append(out)
156             continue
157         print(f"[fusion_golden] ({vi}/{len(specs)}) {spec['name']}: extracting...",
flush=True)
158         t0 = time.monotonic()
159         if telem is not None:
160             telem.event("video_start", spec["name"])
161         try:
162             data = extract_video(spec, frame_skip=frame_skip, iou=iou, telemetry=telem)
163         except Exception as e: # noqa: BLE001 - record machine state at death, then re-
raise
164             if telem is not None:

```

20. user (2026-06-21T09:25:35.478Z)

```

300         + MinIO (via ``endpoint_url``); ``gcs`` = Google Cloud Storage; ``gdrive`` = a
locally MOUNTED
301         Google Drive (``gdrive.mount_root``). Per-backend settings are loose dicts passed to
the constructor ?
302         **secrets are never stored here**, only endpoints / regions / file paths / source-
selectors
303         (boto3/google auth chains supply credentials). See ``backend/storage/``.
304         backend: Literal["local", "s3", "gcs", "gdrive"] = "local"
305         # Output root for produced artifacts. "" => fall back to output.output_dir (non-
breaking).
306         output_root: str = ""
307         # --- Input side (COMPUTE_DECOUPLED_SERVING M2; parallel to the output
`backend`/`output_root`).
308         # Default ``local`` / "" keeps today's behaviour: a bare path / ``local://`` URI is
read IN PLACE
309         # (identity, no copy). Set these to read inputs from a bucket / mounted Drive ? a
bare/relative
310         # input name is resolved against ``input_root`` (e.g.
``input_root``="gs://bucket/videos"`` + "x.mp4"
311         # => ``gs://bucket/videos/x.mp4``) or, if ``input_root`` is empty, prefixed with
``input_backend``
312         # (``"gcs"`` + "bucket/x.mp4" => ``gcs://bucket/x.mp4``). An explicit scheme on the
input
313         # (``gs://?``/``s3://?``) or an absolute local path ALWAYS wins. Non-local inputs
are fetched to a
314         # scratch dir before processing (see ``backend.storage.acquire_local_input``).
315         input_backend: Literal["local", "s3", "gcs", "gdrive"] = "local"
316         input_root: str = ""
317         # --- Per-video workspace (docs/PER_VIDEO_WORKSPACE.md) ---
318         # One folder per video keyed by the MD5 video_id:
<root>/[<workspace_prefix>]/<video_id>/...
319         # so a local deploy and a cloud (bucket) deploy share ONE relative layout (only the
root
320         # differs). DEFAULT-OFF: convergence is phased + flipped via a Launch Report (D5).
When OFF
321         # the legacy flat layout is used unchanged.
322         single_folder_per_video: bool = False
323         # Durable workspace root URI. "" => precedence: output_root, then output.output_dir.
324         workspace_root: str = ""

```

```

325         # Cloud namespace prefix under the root so per-video folders sit tidily beside
326         # videos/ labels/ weights/ (e.g. gs://bucket/workspaces/<video_id>/). "" = root
directly.
327         workspace_prefix: str = "workspaces"
328         local: dict[str, Any] = Field(default_factory=dict)
329         s3: dict[str, Any] = Field(default_factory=dict)
330         gcs: dict[str, Any] = Field(default_factory=dict)
331         gdrive: dict[str, Any] = Field(default_factory=dict)
332
333
334     class DeploymentConfig(BaseModel):
335         """Deployment profile ? one switch that selects a coherent bundle of compute +
storage
336         knobs (COMPUTE_DECOUPLED_SERVING M1, ``docs/COMPUTE_DECOUPLED_SERVING/README.md``
$4.3).
337
338         **Pure / additive:** the empty default profile (``""``) applies NO preset, so the
pipeline
339         behaves exactly as before this section existed. A profile is opt-in (``config.json``

```

21. assistant (2026-06-21T09:25:39.320Z)

Now let me check `serve\_contrast.py` (likely the owned-vs-Gemini shadow comparison the \$4.6 step-3 refers to) and confirm there's no existing consent field anywhere, and check how the Gemini reel/rally output is produced/stored today (the capture inputs).

22. user (2026-06-21T09:25:41.527Z)

```

=== serve_contrast head (owned-vs-Gemini?) ===
"""The eval?serve worked example: score a cue at BOTH windowings and run the promotion guard.

```

This is the reproducible proof of the foundational fix. For the Gate-A held-shuttle guard it builds candidate windows directly from each golden trajectory CSV under BOTH named windowings (:data:~backend.eval.windowing.PROPOSAL` and :data:~backend.eval.windowing.SERVED`), scores CONTROL vs Gate-A (``net\_crossings >= 2``) at each, and feeds the per-video held-out ?F1 into :func:~backend.eval.promotion.promote_if_served_safe`.

It demonstrates the bug *and* the fix in one run:

- PROPOSAL windowing: Gate-A wins (historically **?F1 ? +0.074**, CI clears 0, 6/0 sign test).
- SERVED windowing: Gate-A does **not** win (historically **?F1 ? ?0.008**, real-rally loss).
- The promotion guard therefore **vetoes** a promotion that would otherwise pass on the proposal numbers ? a cue cannot be promoted on a windowing production doesn't serve.

Unlike the persisted ``\*\_golden\_features.json`` (which are PROPOSAL-windowed only), this re-windows from the raw trajectory CSVs so the SERVED candidate set is genuinely different. Pure CPU, GPU-free, no Gemini, no DB. Reuses ONLY shipped pieces (windowing, rally_gate, metrics, promotion). Owner runs it against the real golden CSVs; tests exercise it on synthetic trajectories.

CLI::

```

python -m backend.eval.serve_contrast --manifest output/human_lovo_manifest.json
"""
from __future__ import annotations

import argparse
import json
from typing import Any, Dict, List, Sequence, Tuple

from backend.eval import windowing as W
from backend.eval.calibrate_wasb import load_golden_gts
from backend.eval.gemini_refine import merge_overlaps
from backend.eval.metrics import score
from backend.eval.promotion import promote_if_served_safe

```

```
from backend.pipeline.detectors import rally_gate
from backend.pipeline.detectors.tracknet_runner import TrackNetRunner
```

```
Interval = Tuple[float, float]
IOU = 0.5
```

```
=== any consent/attestation field anywhere in backend ===
backend/eval/gt_loader.py
```

23. assistant (2026-06-21T09:25:45.857Z)

No consent field exists yet anywhere except `gt\_loader.py` (let me confirm that's unrelated). Let me also confirm how `compare()` maps onto an "owned-vs-Gemini" shadow-eval ? the experiment harness compares two `Variant`s, both of which are offline re-scoring recipes. The Gemini output is a different shape (it's a serving output, not a persisted candidate set). This is a key seam distinction. Let me verify what `gt\_loader` consent reference is and check the dual-eval-set wiring.

24. user (2026-06-21T09:25:50.583Z)

```
=== gt_loader consent context ===
11:quantity the ship-gate depends on (gt_hash, LOVO scores, the future consent fingerprint)
```

```
=== dual-eval-set in regression? ===
```

```
=== how is a Gemini reel/rally output represented (intervals/report)? ===
backend/pipeline/segmenters/gemini.py
backend/pipeline/segmenters/trajectory_hybrid.py
backend/pipeline/segmenters/yolo_hybrid.py
backend/pipeline/segmenters/__init__.py
backend/pipeline/stitcher/compiler.py
--- workspace single_folder wired into any write site yet? ---
backend/config/models.py
```

25. assistant (2026-06-21T09:25:57.528Z)

Key confirmations:

- `single\_folder\_per\_video` workspace helper is **default-OFF and NOT wired into any write site** (only referenced in `models.py` config). So the capture/store step has a pure helper but no production write path yet.
- `gt\_loader.py` already anticipates a "future consent fingerprint" ? confirming consent is a planned-but-unbuilt data field.
- The experiment harness `compare()` operates on two offline `Variant` re-scoring recipes over persisted `VideoCandidates`; the Gemini reel is a serving output (intervals/report), not yet a `VideoCandidates`. So a shadow-eval "owned-vs-Gemini" adapter needs an ingest shim that turns a Gemini rally output + an owned-model rally output into the harness's interval shape.

Let me confirm the segmenter output shape (intervals/report) and how rally output is produced, to nail the capture-input shapes precisely, and check the experiment harness tests to confirm what's mock-testable.

26. user (2026-06-21T09:26:01.078Z)

```
=== segmenter base output shape ===
3:from backend.infrastructure.database import Database
4:from backend.results import Result # new standardized contract (Result[Value] ==
Success[Value] | Failure)
6:class BasePlaySegmenter(ABC):
24:     def process_video(self, video_id: str, video_path: str, player_pool: List[str] = None,
max_frames: int = None) -> "Result[List[Dict[str, Any]]]":
27:         and returns a Result.
31:         failure, etc.). This is the start of the standardized error/result contract.
35:class SegmenterRegistry:
37:     self._segmenters: Dict[str, Type[BasePlaySegmenter]] = {}
```

```

39:     def register(self, segmenter_cls: Type[BasePlaySegmenter]):
40:         """Registers a new segmenter class under its name property."""
41:         raise ValueError(f"Failed to resolve name property from segmenter class
42: {segmenter_cls.__name__}: {e}") from e
43:     def get(self, name: str) -> Optional[Type[BasePlaySegmenter]]:
44:         return self._segmenters.get(name)
45:     return res
46: segmenter_registry = SegmenterRegistry()

```

```

=== experiment harness tests (mock-testable surface) ===
test_experiment.py
test_experiment_honest.py
test_fusion_golden_telemetry.py
test_golden_fixtures.py
test_golden_fixtures_split.py
test_golden_real_fixtures.py
test_workspace.py
./claude/worktrees/agent-a771ff1d85e18f2ed/tests/test_experiment.py
./claude/worktrees/agent-a771ff1d85e18f2ed/tests/test_experiment_honest.py
./claude/worktrees/bhi-execpolicy/tests/test_experiment.py
./claude/worktrees/bhi-execpolicy/tests/test_experiment_honest.py
./claude/worktrees/boxhill17/tests/test_experiment.py
./claude/worktrees/boxhill17/tests/test_experiment_honest.py
./claude/worktrees/gap-doc/tests/test_experiment.py
./claude/worktrees/gap-doc/tests/test_experiment_honest.py
./claude/worktrees/hardchop/tests/test_experiment.py
./claude/worktrees/hardchop/tests/test_experiment_honest.py
./tests/test_workspace.py

```

27. assistant (2026-06-21T09:26:11.954Z)

I have enough grounding. There are existing test files for `experiment` and `workspace`, confirming both surfaces are unit-testable in pure/offline mode. Let me do one final check on how the segmenter result (intervals) gets persisted to confirm the capture inputs exist as durable artifacts, and confirm there's no existing "promote to golden TRAINING set" production code path (only the offline `fusion\_golden` extractor exists).

28. user (2026-06-21T09:26:16.619Z)

```

=== where rally intervals/report get written today (capture inputs) ===
266:         f"SKIPPED WITHOUT ANALYSIS ? segments are missing for those time ranges. "
267:         f"SKIPPED WITHOUT ANALYSIS ? 0 segments here does NOT mean 'no rallies'."
268:     logger.info(f"{len(validated)} play segments passed validation (out of
269: {len(ai_segments)} from Gemini).")

=== is there ANY production goldenize/promotion-to-training code (vs offline fusion_golden)? ===
backend/config/models.py
backend/eval/eval_stats.py
backend/eval/golden_real_fixtures.py
backend/eval/per_user_gate.py
backend/eval/per_user_regression.py
backend/eval/promotion.py
backend/eval/rally_seg_eval.py
backend/eval/serve_contrast.py
backend/eval/tolerance_metrics.py
backend/eval/training.py

=== promotion.py (the gate) ===
4:**Why this exists.** ``per_user_gate.should_promote_for_user`` already grades a B?A lift
13:``should_promote_for_user``:
24:paths are untouched until a caller routes its A/B through :func:`promote_if_served_safe`.
32:from backend.eval.per_user_gate import PromotionDecision, should_promote_for_user
34:__all__ = ["ServedGateResult", "promote_if_served_safe", "served_regresses"]
37:def served_regresses(a_scores: Sequence[float], b_scores: Sequence[float], *,
78:    - ``promote`` ? final verdict. ``True`` only when the underlying honest gate would

```


promote

```
81: - ``vetoed_by_served`` ? ``True`` when the proposal/base gate WOULD have promoted but the
83: - ``base_decision`` ? the ``PromotionDecision`` from ``should_promote_for_user`` on the
84:   *promotion-grade* scores (served by default ? see :func:`promote_if_served_safe`).
```

29. assistant (2026-06-21T09:26:29.242Z)

I now have full grounding. Key findings confirmed:

- `**`promotion.py` / `per_user_gate.should_promote_for_user`**` already implement the `**promote-if-served-safe` gate (the goldenize `*decision*` logic) ? pure, offline, unit-tested. The flywheel "promote good ones to golden" can reuse this gate; what's missing is the `*write a golden TRAINING row*` sink (the production analogue of the offline ``fusion_golden`` extractor) plus the human-review step.
- `**No production goldenize/promotion-to-training write path exists**`; only the offline ``fusion_golden.run()`` extractor writes ``*_golden_features.json``.
- `**Capture inputs (source upload, Gemini reel, rally output/report)**` are all already produced as durable artifacts today; the workspace helper (``backend/utils/workspace.py``) gives the per-video layout but is `**not wired into any write site**` (default-OFF).
- `**The consent field does not exist anywhere**` ? ``gt_loader.py:11`` even names it a "future consent fingerprint." Consent capture/attestation/retention-routing is the genuine owner+counsel gate (D10/M9), distinct from plumbing.

This cleanly splits into two gates: `**M11-plumbing**` (Claude-doable now, default-OFF, faked inputs) and `**M11-consent**` (owner + counsel, the D10/M9 decision).

Now I'll emit the structured output.

30. user (2026-06-21T09:27:09.602Z)

Structured output provided successfully

31. assistant (2026-06-21T09:27:22.278Z)

Structured output delivered. Summary of what I found in my area (M11 data-flywheel harness + D10/D11 consent gates):

`**Two cleanly-separable gates:**`

1. `**m11-plumbing**` (high unblock value, Claude-doable now): The entire mechanical loop ? capture ? reliably-store ? shadow-eval ? goldenize-stub ? can be built NOW as one isolated default-OFF PR with faked inputs and mocked/storage-fake tests. The seams already exist:
 - Capture/store reuses the pure ``backend/utils/workspace.py::for_video`` helper (default-OFF, currently un-wired into any write site) + the ``StorageBackend`` ABC.
 - Shadow-eval reuses ``backend/eval/experiment.py::compare`/`compare_unlabeled`` ? but the real seam is an `**ingest adapter**`: ``compare()`` consumes ``VideoCandidates`` from persisted candidates, whereas a Gemini reel is a `*serving*` output, so the codeable piece is the adapter mapping both serving outputs into the harness's ``Interval`/`VideoCandidates`` shape (no new scoring math).
 - Goldenize-promotion reuses the existing served-safe gate ``backend/eval/promotion.py::promote_if_served_safe``; the only missing sink is a production analogue of the offline ``backend/eval/fusion_golden.py:149`` golden-row writer, gated behind a faked ``consent_attested=True`` precondition.

2. `**m11-consent**` (medium unblock value, owner+counsel-gated): The DECISION ? D10 counsel-reviewed ToS, adults-only attestation, derived-data/auto-delete-raw retention, and M9 consent-gate wiring ? is genuinely not Claude-doable. No consent field is wired anywhere today (``backend/eval/gt_loader.py:11`` even calls it a "future consent fingerprint").

`**The precise seam:**` the consent `*field*` can be wired inert/faked now (defaulting to the no-consent auto-delete branch, matching today), so the consent?capture vs no-consent?delete fork is fully testable before the real consent SOURCE exists. Only the real consent source, the legal/policy text, and routing real user raw video into the training pool require the owner/counsel gate.

Note: `git pull --ff-only` failed at session start (the local branch `feat/serving-m4-device-context` tracks a remote ref that wasn't fetched) ? I proceeded read-only on the local checkout, which is even with `origin/master` plus the two shipped M4 commits, so the analysis reflects current remote truth.